

I H A T E T O B D G E T

**A simple web app to understand and control your expenses.
Designed to be self-hosted.**

Frequently Asked Questions (FAQ)

Since IHateToBudget is fundamentally a self-hosted platform, the distinction between the "end-user" and the "developer/administrator" is heavily blurred. The person logging daily expenses is almost always the same person spinning up Docker containers and adjusting configuration keys. This FAQ explicitly answers both technical deployment roadblocks and frontend user behavior introduced by the Bootstrap 5 upgrade.

I. Deployment & Administration FAQs

Q. Is this application intended to be hosted securely on a public network?

A: While a basic authorization and login system exists, IHateToBudget is primarily engineered to be self-hosted within a local home network (e.g., on a home server or Raspberry Pi) and utilized by a small pool of trusted household users. If exposing it to the open web, wrapping it inside a strict VPN (like Tailscale or WireGuard), or adding an extra layer of basic auth via your reverse proxy is recommended.

Q. My recurring transactions are not updating automatically. What is missing?

A: The application relies on an internal system cron to cycle recurring expenses. When launching or restarting your environment, you must manually kickstart the cron task within the active app container by executing:

```
docker-compose exec ihatetobudget service cron start
```

Q. Where is my data kept, and how can I safely run backups?

A: All tracking sheets, expense indices, and categories are saved in a local SQLite file (db.sqlite3) generated in your repository root directory. To back up your financial history safely, drop the containers temporarily and copy the file:

```
docker-compose down -v  
cp db.sqlite3 db.sqlite3.bak  
docker-compose up -d
```

Q. I managed to lock myself out of the interface. How do I generate a new administrator login?

A: You can use the Django CLI framework inside an ephemeral container environment to register a new superuser directly into the active database volume by running:

```
docker-compose run --rm ihatetobudget pipenv run python  
manage.py createsuperuser
```

Q: The container is throwing a "Bad Request (400)" error when I try to access it via my domain name. How do I fix this?

A: Django has built-in security that blocks unlisted hostnames. You need to edit your configuration to ensure your domain or local IP is explicitly added to the ALLOWED_HOSTS variable (e.g., ALLOWED_HOSTS=localhost, 127.0.0.1, mybudget.local).

Q: How do I export my budgeting sheets if I want to migrate away or view them in Excel?

A: Since the data lives in SQLite, you can use an external tool like sqlite3 or DBeaver to connect to your local db.sqlite3 file and dump tables to CSV. Alternatively, you can run a native Django management command within the container to dump the entire database to a clean JSON file:

```
docker-compose run --rm ihatetobudget pipenv run python  
manage.py dumpdata > budget_backup.json
```

Q: Why are my charts showing incorrect dates or times for expenses?

A: By default, the Docker container uses UTC. If your local time zone doesn't match, your transactions might register on the wrong day. You can fix this by adding a timezone environmental variable (TZ=Asia/Kuala_Lumpur or your respective zone) to the environment: block of your web service in docker-compose.yml.

II. Bootstrap 5 Migration & Interface FAQs

Q. Why migrate to Bootstrap 5 if the original upstream repository was archived?

A: The upstream codebase was built on older design systems relying heavily on legacy styling and jQuery dependencies. This Bootstrap 5 migration brings the layout forward into a contemporary web stack. This dramatically improves native mobile responsiveness (for log-as-you-go use cases) and ensures security compliance with modern web frameworks.

Q. Will migrating my old deployment to this Bootstrap 5 version break my existing data models?

A: No. The core Django database schema remains completely backward compatible. When pulling down the bootstrap5-full-migration codebase, your local sheets remain untouched. You only need to run the structural database migrations to match the latest system definitions:

```
docker-compose run --rm ihatetobudget pipenv run python manage.py migrate
```

Q. How can I change the base currency symbols to match non-Euro regions?

A: Currency styling rules are controlled globally via environmental keys. Open your docker-compose.env config file and redefine the localization strings to point toward your region. For example, US Dollars, apply the following block:

```
CURRENCY_GROUP_SEPARATOR=,  
CURRENCY_DECIMAL_SEPARATOR=.  
CURRENCY_PREFIX=$  
CURRENCY_SUFFIX=
```

Q: Does this Bootstrap 5 migration include a Dark Mode toggle?

A: Bootstrap 5 introduces native dark theme utilities (data-bs-theme="dark"). If you haven't implemented a toggle button yet, you can explain if it automatically honors the user's system preference via CSS media queries, or if it is a planned feature on the migration roadmap.

Q: I modified a CSS or JavaScript asset file, but the changes aren't reflecting in my browser. Why?

A: Django caches static assets heavily in production configurations. After updating design files, you need to tell Django to collect and rebuild the static asset pipeline by running:

```
docker-compose run --rm ihatetobudget pipenv run python manage.py collectstatic --noinput
```